百亿补贴 C 端 AI Coding 实战：端到端 CodingAgent 设计与实践

原创 天猫技术团队 大淘宝技术 2026年8月2日 23:34 浙江 19人



本文介绍了百亿补贴 C 端 AI Coding 实战中端到端 CodingAgent 的设计与实践，旨在解决前端开发中重复编码、上下文理解成本高及视觉还原难等痛点。该 Agent 采用基于规范驱动与知识增强的自反思架构，通过构建包含页面、模块、组件等多层级的垂直领域知识库，并结合 Git Hooks 实现知识库的自动化同步与在线进化，确保代码生成的业务准确性与时效性。其核心能力涵盖多粒度输入适配、仓库智能匹配、设计稿到代码的智能转换（D2C）、业务代码生成及多层次质量验证，最终实现从需求描述到可交付工程代码的端到端自动化闭环，并已在依赖升级、页面开发等实际场景中落地提效。

01 引言：垂直领域 AI 编程的挑战与机遇

在前端工程领域，ToC 业务开发者长期面临这几个结构性问题：

- **重复性编码占比高**：大量页面、模块的结构高度相似，但每次都需要重复的接口调用、字段映射、样式还原...
- **上下文理解成本大**：业务仓库日益庞大，新需求开发前的"代码考古"时间不断增长，理解组件用法、定位依赖关系等琐碎工作通常占据了大量的开发时间。
- **视觉还原是持续痛点**：C 端业务的视觉还原是永远绕不开的话题，经常因为几像素的差值反复调整...
- **知识沉淀难以复用**：团队积累的组件知识库分散且缺乏长期维护，知识库文档的置信度很差。

随着近期 AI 模型的发展，AI 在**代码补全**、**通用编程辅助**、**图像分析**等领域带来的结果稳定性和飞速编程体验不断突破了传统编程模式，但回归垂直领域，这些工具仍无法直接实现业务的端到端交付。

本文致力于构建一个深度绑定百补业务域的 coding-agent，让它真正理解我们的技术体系并完成端到端的业务交付。

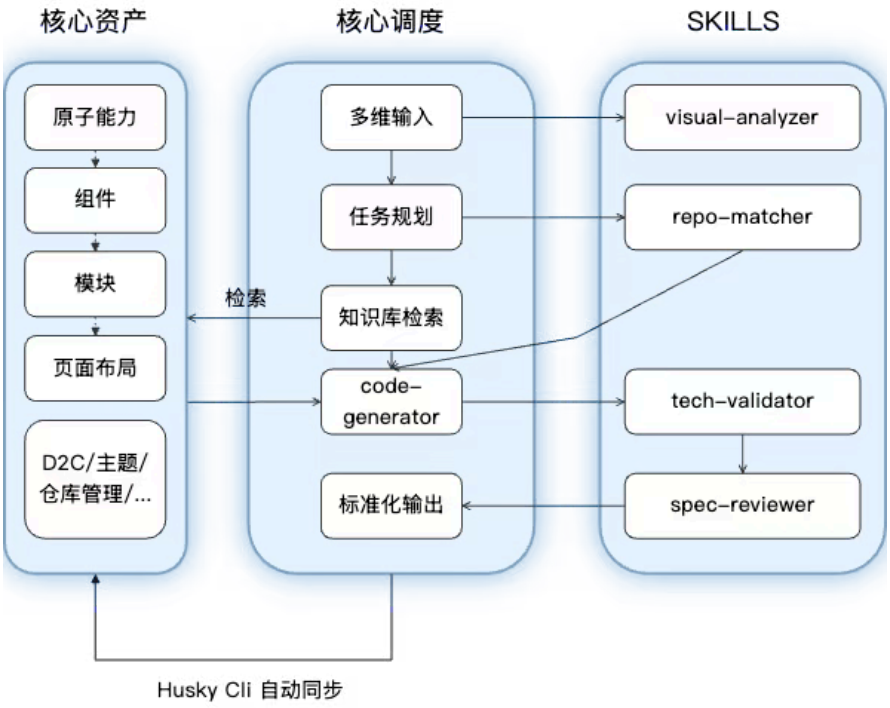
- **多维输入**：支持从一句话描述到完整 PRD 的多种粒度输入。
- **输出可交付**：产出的不是代码片段，而是可直接进入业务迭代流程的工程代码。

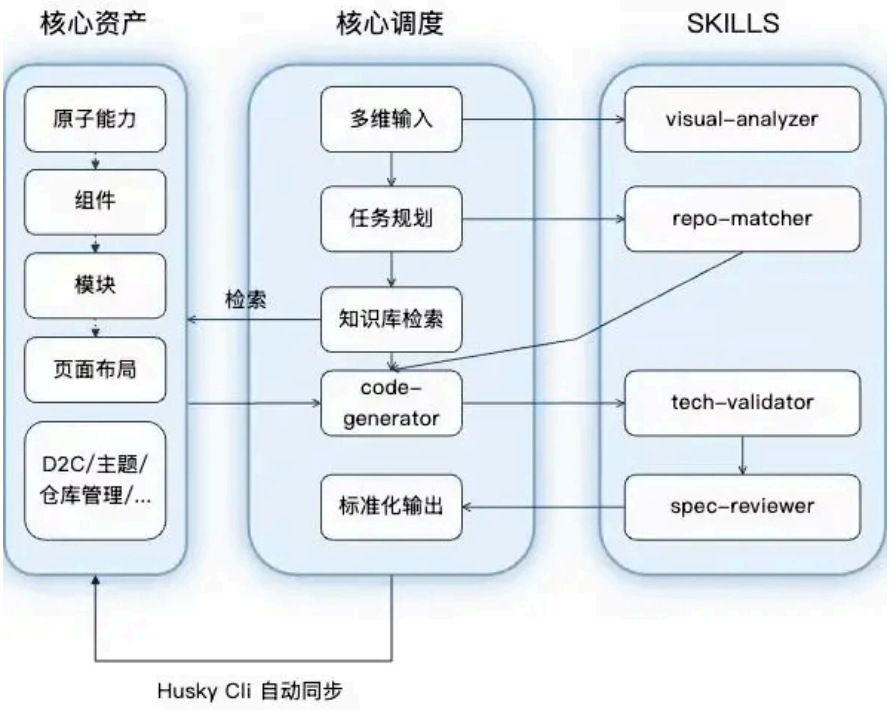
- **知识库驱动**：以垂直领域知识库为核心引擎，保障生成代码的业务准确性和规范一致性。
- **持续进化**：知识库与日常迭代深度绑定，Agent 随需求一起成长。

02 整体架构：规范驱动的自反思智能体

垂直领域的端到端交付，Agent 核心的价值不仅仅在于面向需求给出技术方案和代码实现细节，更应该在交付时保障代码质量和可迭代性。基于此，我们提出了整体的架构设计理念：

> 基于规范驱动（Specification-Driven）与知识增强（Knowledge-Augmented）的自反思（Self-Reflective）智能体架构





规范驱动：物料资产体系

通过总结百补现阶段业务迭代场景的模式，我们建立了完整的物料资产体系：

1. 将页面元素划分为颗粒度更小的**模块**、**组件**、**原子能力**、**主题**等物料资产，并为每类资产提供完整配套的知识库（含 Props 定义、示例场景、迭代记录等）。这种结构化的知识组织方式，使得知识库 + Agent = 合适的组件选择和示例代码，大大降低了组件的上手成本。
2. 构建了页面 **Solution**（页面解决方案）作为页面布局框架、数据输入（包括 SSR）、事件通信、用户交互的底层基座。Solution 提供了代码**语法糖**，将页面级别的工程开发降级为模块级、组件级，大大减少了上下文之间的耦合度。

知识增强：自动化同步机制

知识库作为资产核心，面临一个关键挑战：在多人协作、频繁迭代的 C 端场景下，如何保证数据的准确性和完备性？如果知识库的迭代无版本化管理，`coding-agent` 将无法给到符合预期的答案。

为了解决这个问题，我们针对核心知识库和业务知识库分别做了以下设计：

- 为核心知识库构建了 `llm-doc-async-agent` 服务：通过配置 husky 在 commit 阶段自动同步代码变更到知识库，保证每一次迭代带来的变更行为都能在知识库中同步更新，从根本上消除了开发者维护组件知识库的心智负担。
- 为业务知识库更新研发了 `knowledge-base-updater` 技能：用于Agent 应用时在线化更新业务信息，保证每一个业务需求开发阶段的信息共享，缓解了开发者维护业务知识

库的压力。

自反思：质量保障闭环

`coding-agent` 明确约束了产物的输出不仅仅是代码，还包括：

- **变更日志**：用于纠错总结，通过对比变更日志与技术规划/需求分析进行循环纠错
- **结构化输出**：依赖版本升级信息（beta or latest）、组件使用记录等，便于追溯和审查

03 Agent 核心能力：模块化设计与技能编排

面向多场景的输入适配

输入形态	适用场景	示例
一句话描述	简单需求、快速原型	"参考页面 **A**，快速实现一个页面 **B**"
多轮对话	需求不明确，需要交互式澄清	"倒计时结束的回调需要触发埋点上报并刷新接口"
设计稿 / 图片	视觉驱动的 **D2C** 场景	上传 **MasterGo** 链接和截图，生成对应模块的代码
完整 **PRD**	复杂业务需求，完整交付	上传需求文档，端到端生成全部代码

Agent 调度中枢与决策机制

Agent 作为整个系统的调度中枢，核心职责是获取用户的输入和工作空间信息，并确认以下关键信息来决策后续流程：

决策维度	可选项	说明
任务模式	本地任务 / 远程任务	决定代码执行环境
任务范围	需求变更 / 代码优化	决定了任务完整度
任务类型	页面开发 / 组件开发 / 混合任务 / 用户答疑	决定调用的 **Skill** 组合
系统偏好	先提示后开发 / 直接开发 / 是否需要代码推送	决定交互模式

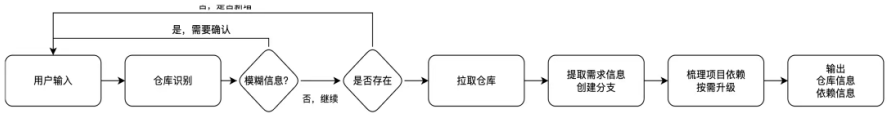
SKILLS 技能体系：场景驱动的能力封装

- repo-matcher：仓库匹配与依赖治理

构建可用于代码开发的工作环境

方案设计

`repo-matcher` 通过知识库中的仓库元信息，结合用户输入的需求描述进行语义匹配：



核心能力

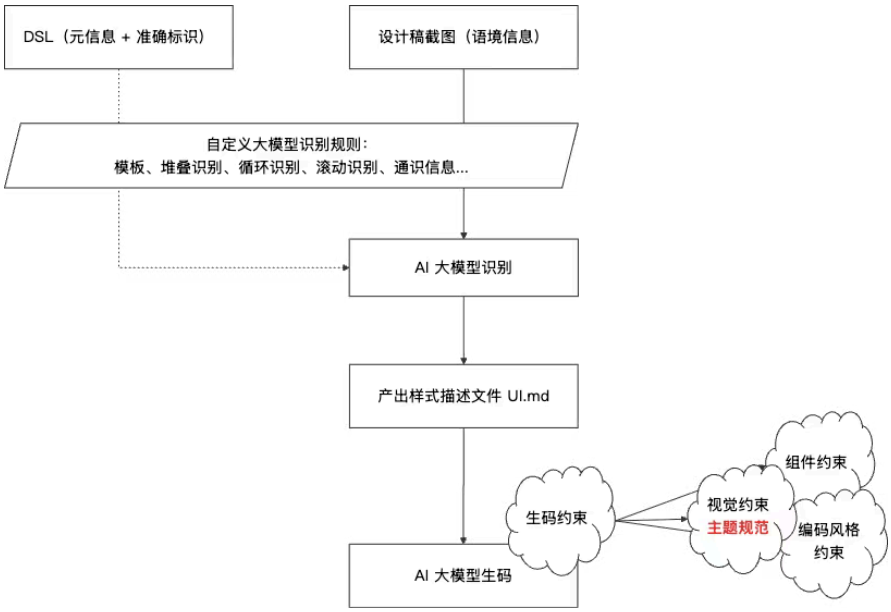
- **仓库智能匹配**：基于需求描述自动识别目标仓库，避免手动查找
- **分支规范创建**：按照团队规范自动生成分支名，避免中文分支等异常
- **依赖版本治理**：检查 beta/latest 版本策略，识别潜在的 break change 风险

- visual-analyzer：设计稿解析与视觉还原

D2C 场景的优化方案，提高视觉还原可接受度，降低视觉高还原的重复性工作

方案设计（详见章节：《AI-D2C 能力建设：从设计稿到代码的智能转换》）

`visual-analyzer` 采用 `结构化数据 + 多模态还原 + 领域 DSL` 的三层架构：



核心能力

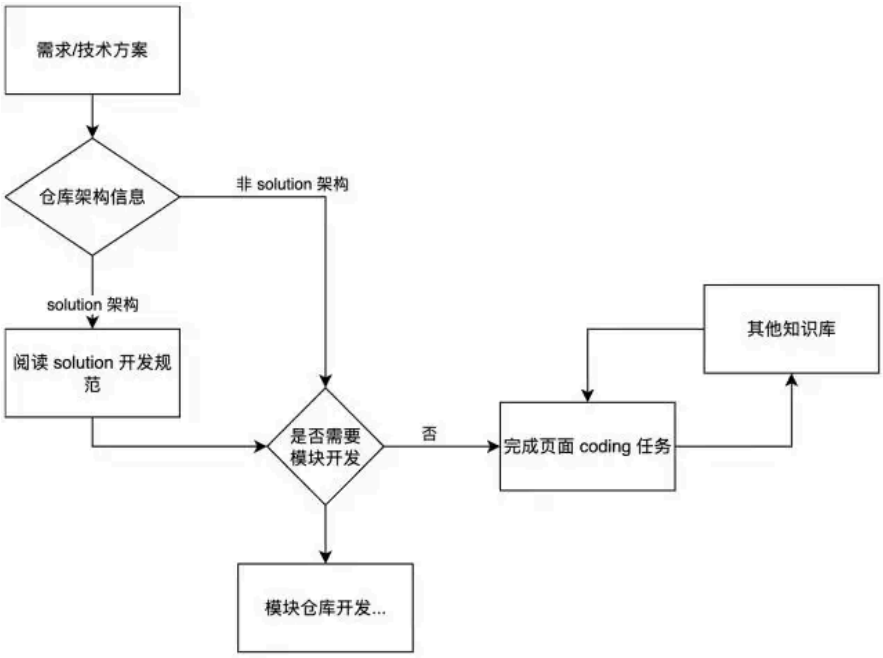
- **设计稿结构解析**：通过 MCP 工具获取 MasterGo 设计稿的 schema 信息
- **多模态布局验证**：结合截图信息消除多图层噪音干扰
- **组件智能路由**：基于 UI 特征 DSL 自动匹配标准化组件

- code-generator：业务代码智能生成

vide coding 的核心，阅读仓库代码、知识库加载、业务逻辑编写

方案设计

`code-generator` 基于知识库中的模板和示例，结合仓库框架进行代码生成：



核心能力

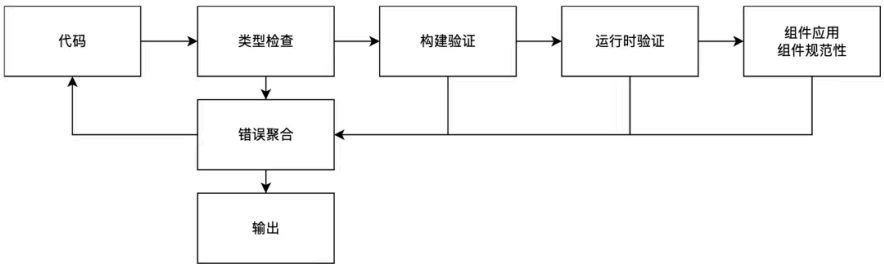
- **页面级开发**：基于 Solution 框架生成完整页面结构
- **模块级开发**：生成业务模块代码，处理数据流和交互逻辑
- **组件级开发**：生成或组合标准化组件，确保 API 调用正确

- tech-validator：代码质量自动化验证

用于解决 vibe coding 通用型问题 – 代码可维护性、代码可读性、代码整洁度

方案设计

`tech-validator` 构建了多层次的验证体系：



核心纠错场景

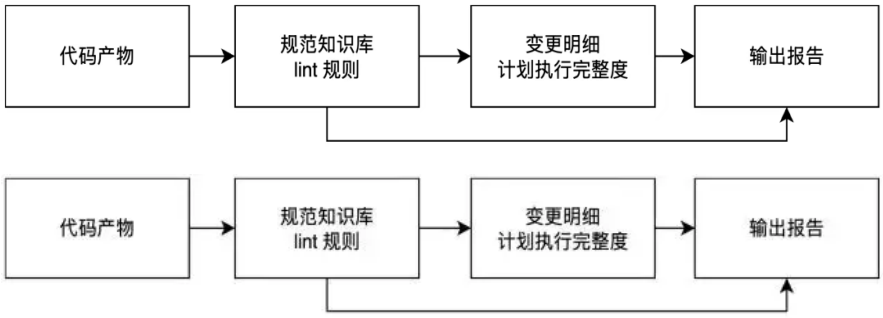
- 类型检查：TypeScript 类型验证，确保类型安全
- 构建检查：编译错误检测，提前发现问题
- 运行时检查：模拟运行验证，检测运行时异常
- 组件规范检查：是否合理应用组件、验证组件调用是否符合团队规范

- spec-reviewer：规范审查与变更追溯

用于解决 vibe coding 通用型问题 – 代码工程化、组件使用模式、需求完成度

方案设计

`spec-reviewer` 基于规范知识库进行逐条检查：



核心能力

- **规范逐条检查**：依据团队规范检查代码质量
- **结构化报告**：输出可读性强的审查报告
- **变更日志生成**：记录本次开发的所有变更，便于追溯

- knowledge-base-updater：知识库自我进化与在线学习

在实际使用中，业务场景不断扩展，知识库无法覆盖所有情况：

- 新业务上线（如「场景秒杀」），知识库中没有对应的组件和模式

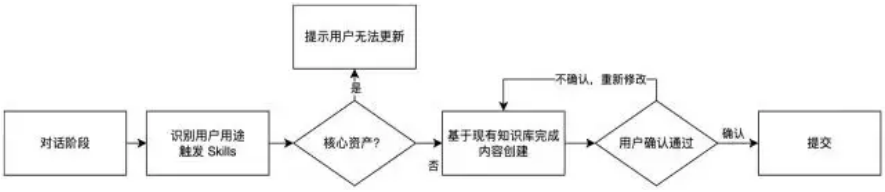
- 用户 A 在迭代中积累了宝贵的业务经验，但无法自动沉淀
- 用户 B 遇到类似场景时，Agent 无法复用已有经验
- 依赖人工维护知识库，效率低且容易遗漏

设计方式

`knowledge-base-updater` 利用知识库托管 Git 仓库的特性，实现 Agent 的在线化知识更新：

核心流程

1. 新知识识别：在对话过程中，识别用户描述的新业务场景、新组件用法、新的最佳实践
2. 结构化提取：将非结构化的对话内容转换为符合知识库格式的结构化条目
3. 冲突检测：检查新知识是否与现有知识冲突，避免覆盖已有内容
4. 安全提交：通过 Git 提交变更，保留完整的修改记录
5. 即时生效：下次对话时自动拉取最新知识库



安全机制

知识库的自动更新带来了信息安全和恶意篡改的风险，我们通过以下机制保障安全：

安全维度	实现方式	说明
变更追溯	依托 **Git** 天然支持	每次知识更新都有完整的 **commit** 记录，可追溯到具体用户和时间
回滚能力	依托 **Git** 天然支持	发现问题可快速回滚到任意历史版本
白名单配置	**SKILL** 内部支持安全白名单文件	原理类似于 **.gitignore**，可配置仓库的核心资产

核心能力

- 新知识自动识别：从对话中提取有价值的新业务知识
- 结构化入库：将非结构化内容转换为标准知识格式
- 增量更新：只更新变化的部分，避免全量覆盖
- 安全审计：完整的变更记录，支持追溯和回滚
- 跨用户共享：一次沉淀，全员受益

设计动机：为什么需要分层

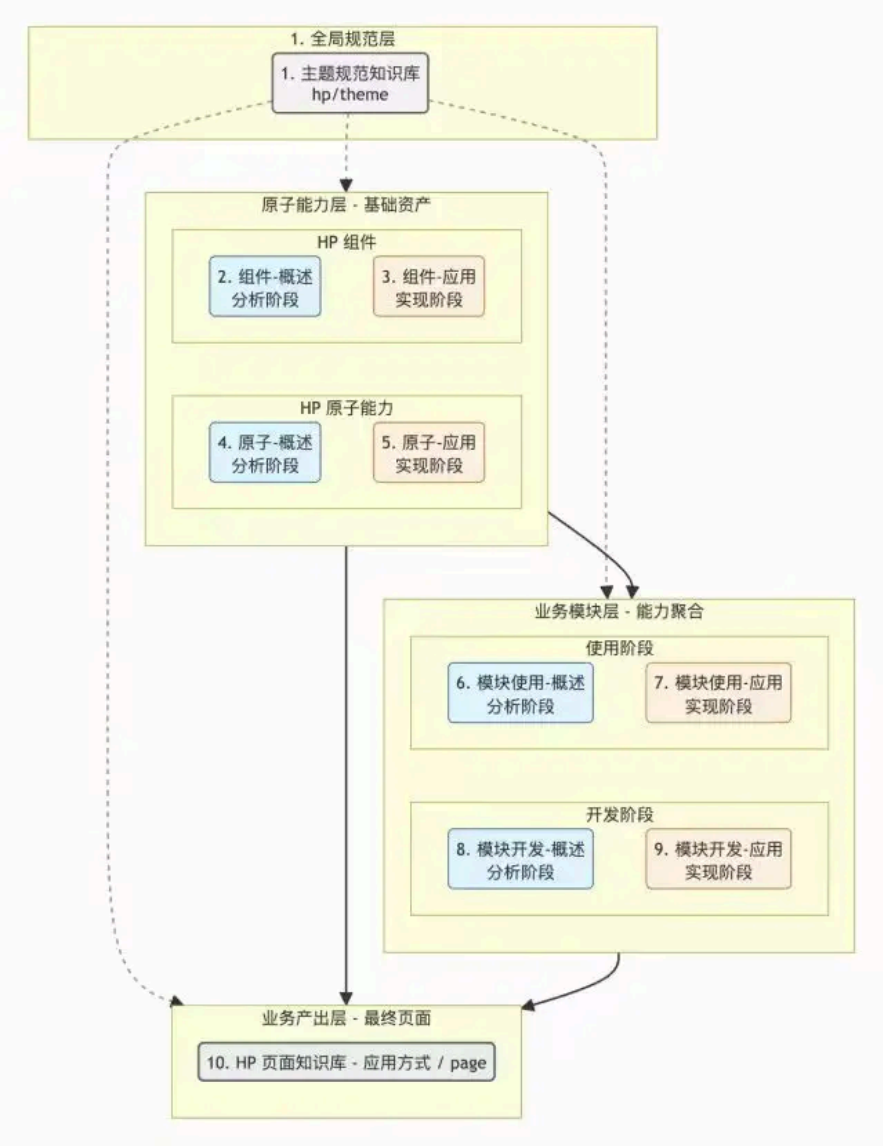
在构建 `coding-agent` 的过程中，我们发现一个核心问题：**不同粒度的开发任务需要不同层级的知识支撑**。

- 开发一个完整页面时，Agent 需要理解页面骨架、数据请求模式、模块编排方式
- 开发一个业务模块时，Agent 需要知道具体的业务逻辑、可用的组件、组件之间的组合方式
- 调用一个组件时，Agent 需要准确的 API 文档、使用示例、常见问题

如果把所有知识平铺在一起，会导致：

1. 上下文过载：Token 消耗过大，模型响应变慢
2. 信息噪音：不相关的知识干扰模型判断
3. 检索低效：难以快速定位所需知识

知识库框架设计



- 五层知识体系

层级	内容	作用	索引方式
页面级	**Solution /** 完整页面模板、页面结构模式	**Agent** 理解页面整体骨架、布局、数据请求、事件通信机制	按页面类型索引
模块级	聚焦特定业务定制能力的模块	**Agent** 理解模块的迭代方式、使用场景	按业务场景索引
组件级	组件 **API**、用法示例、**Props** 定义	**Agent** 正确使用组件库，避免重复造轮子	按组件名称索引
原子能力级	工具函数、**Hooks**、基础能力	**Agent** 复用已有基础设施	按功能类型索引
主题设计	设计 **Token**、色值体系、间距规范	**Agent** 生成的代码在视觉上符合设计规范	按设计属性索引

- 知识库内容结构化设计

索引结构设计：

```
<!-- 本文档为"概述索引类"文档，旨在说明在什么场景下需要/必须使用此 npm 包，如果 np
# ${组件名称}
## 概述
<!-- 组件概述信息 -->
## 子目录清单
### ${子目录-1}
```

```
#### 功能清单
<!-- 应用场景摘要，用于匹配功能 -->
#### 视觉规范
<!-- 视觉摘要，用于匹配 UI -->
### ${子目录-2}
#### 功能清单
<!-- 应用场景摘要，用于匹配功能 -->
#### 视觉规范
<!-- 视觉摘要，用于匹配 UI -->
```

内容结构设计：

```
<!-- 本文档为"应用说明类"文档，旨在告诉大模型在给定的实际业务场景下如何合理、正确、
# ${子目录-1}
## 概述
<!-- 应用场景摘要，用于匹配功能 -->
## 安装
<!-- 安装说明，bash 命令 -->
## 规则
<!-- 以下为示例规则 -->
- 熟读 [API 文档](
#API
) 文档，禁止使用未声明的属性。
- 熟读 [代码演示](
#代码演示
)，快速学习组件在组件在实际业务场景下的使用方式并且理解如此设计的原因，当用户需求需要
#代码演示
) 中该组件提供的方案，如果现有方案无法解决实际问题，参考 [API 文档](
#API
) 提供的基础能力进行开发。
- 实现需求时需要仔细地阅读和理解组件 [视觉规范](
#视觉规范
)
  - 如果需要定义组件样式，需要保证定义新的样式与内置样式匹配。
  - 如果需要将组件与其他 React 组件进行组合，需要保证组合后的样式与组件内置样式匹配
  - **重点关注样式如下**: 颜色、背景色、宽度、高度、字号、单位。其他样式需要根据实际
<!-- 其他组件相关的特殊规则需额外定义  -->
<!-- 其它 showcase / badcase 可额外补充 -->
## 代码演示
<!-- 完整的组件代码示例，需明确 props 传入格式和消费场景，便于 AI 理解 -->
## API
<!-- 组件 props 定义、工具方法 fields 字段定义、枚举场景定义；typescript 效果
## 视觉规范
<!-- 基于 ai 分析的视觉，用于 d2c 链路，本章节内容可不补充  -->
```

- 辅助知识库

现有的 AI 模型已经具备了很强的推理能力，在 git、图像分析等场景也学习了大量的领域知识。但为了保证端到端工程化链路的稳定性和准确率，`coding-agent` 还定义了以下辅助知识库，用于规避常见的 `bad case`：

应用场景	内容	作用
repo-matcher	仓库元信息 仓库识别标准、分支切换标准 依赖识别、验证、升级策略	避免"创建中文分支"、"仓库匹配错误"、"依赖升级带来 **break change**"
visual-analyzer	**D2C** 图层识别规范 **D2C** 产物标准	定义领域内通识知识，如移动端 **750** 像素、价格成组、剔除状态栏等

自动化更新：与代码迭代同步

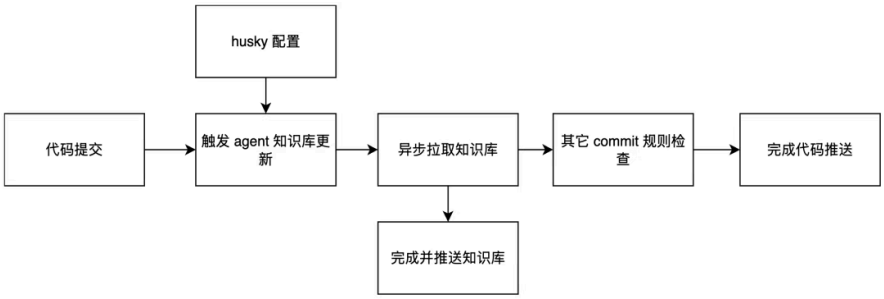
- 痛点分析

知识库最大的挑战是保持时效性。传统的文档维护方式存在以下问题：

1. 依赖人工：开发者在迭代组件后，需要手动更新文档
2. 容易遗忘：业务压力下，文档更新往往被忽略
3. 版本滞后：文档与代码版本不一致，导致 Agent 生成错误代码
4. 质量参差：不同开发者写的文档质量不一

- 解决方案：Git Hooks + LLM 自动同步

我们设计了完全自动化的文档更新机制：



核心实现

```
#!/bin/sh
# 加载 .env 文件中的环境变量
if [ -f .env ]; then
export $(grep -v '^#' .env | xargs)
fi
npx @ali/hp-agent@beta llm-doc-sync "同步并且允许推送（不必询问）"
```

核心能力

1. 开发者提交代码时，Husky 自动触发 `llm-doc-sync` 命令
2. `llm-doc-async-agent` 分析 `Readme.md` 和 `src/*` 的变更内容，自动推送更新到知识库仓库



AI-D2C 能力建设：从设计稿到代码的智能转换

行业现状与挑战

D2C（Design to Code）作为前端领域的重要分支，旨在通过自动化工具将 UI 设计稿直接转换为可运行的前端代码。不同于传统的规则模板还原、深度学习算法识别，现有的 LLM 和多模态输入为 D2C 带来了新的可能性。

- 主流方案对比

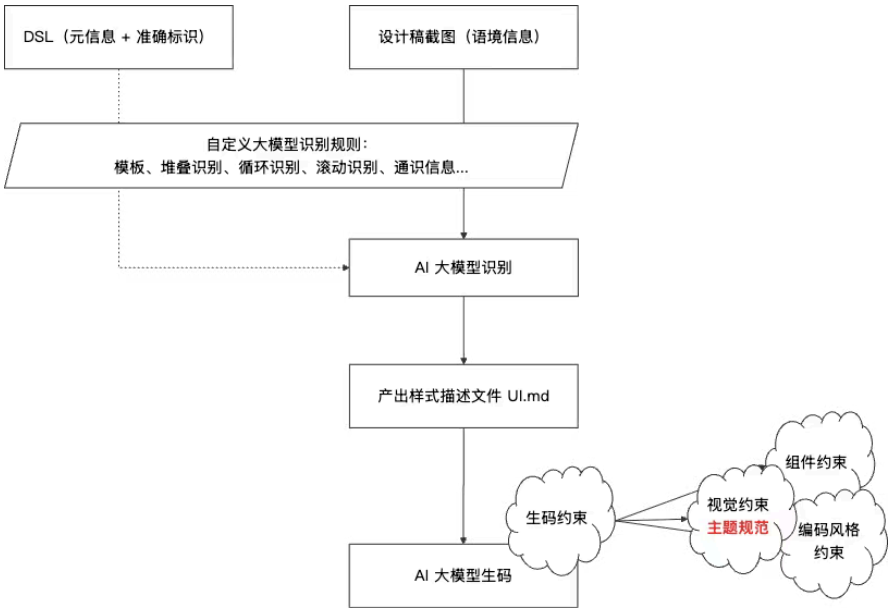
方案类型	代表作品	设计输入形式	优点	缺点
Figma/Sketch 插件式	code.fun / Galileo AI / Anima	Figma/Sketch 设计文件	直接读取设计稿结构，还原度高	依赖设计规范，动效逻辑无法生成
截图转代码	Screenshot to Code / Vercel v0	页面截图	无需原始文件，门槛低	布局语义识别不准，易混淆卡片与列表
自然语言混合生成	Bolt.new / Locofy AI	自然语言 + 参考图	支持"仿照某风格生成"	对中文支持有限，迁移成本高

- 现有方案的核心问题

1. **设计规范依赖**：需要设计师严格遵循 Auto Layout 等规范
2. **语义识别不准**：难以区分卡片、列表、网格等相似布局
3. **组件匹配缺失**：无法与业务组件库关联
4. **Token 消耗过大**：长页面的 schema 数据量巨大

我们的方案：结构化数据 + 多模态还原 + 领域 DSL

- 整体架构



- 三层处理流程

Layer 1：结构化数据获取

通过 MCP 工具直接获取 MasterGo 设计稿的上下文 schema 信息，用于提供精确的数据支撑，包括：图层结构与层级关系、元素属性（位置、尺寸、颜色、字体等）、组件实例与变体信息。

Layer 2：多模态布局验证

提供视觉稿导出的截图信息，用于语境信息的辅助增强：消除多图层累计带来的噪音干扰、验证 schema 解析的骨架布局正确性、处理设计稿与实际渲染的差异。

Layer 3：通过组件特征 DSL 匹配

定义组件的 UI 特征信息，实现组件语意路由：

- 价格组件特征：包含 ¥ 符号 + 数字 + 可选划线价
- 倒计时组件特征：时分秒格式 + 分隔符
- 按钮组件特征：圆角矩形 + 居中文本 + 可选图标

- 实践效果

设计稿截图	还原效果



- 待解决的问题与后续规划

本阶段的 AI-D2C 是一次轻量化的尝试，解决了部分场景下的视觉还原问题，并没有达到1比1的视觉还原效果，且应用场景受模型限制。

现阶段挑战

- 图层噪音：紊乱的 schema 信息干扰业务语义识别，即便通过截图提供了语境信息，重复图层重叠带来的干扰仍无法避免。
- 长页面 Token 消耗：大体积 schema 导致 Token 过量，解析长页面的 Schema 信息往往会出现上下文信息缺失或错乱，目前主要应用于模块级的视觉还原场景。
- 虽然在设计组件库的初衷以及对齐了“设计协议”，然后随着业务需求的迭代以及代码/设计语言之间存在的差异性，并没有真正意义上完成对齐。

后续方案？

未来 D2C 在大模型的加持下一定会达到更加令人惊叹的准确性，回归业务本身，我们应该构建一套完备的产品级组件方案（例如 `material-ui`、`antd` ），做到"设计语言驱动组件渲染"，实现：

- 设计 Token 与代码 Token 的自动映射
- 组件变体与设计变体的双向绑定

06 知识库拓展：垂直场景 Agent 生态

基于完备的知识库体系，知识库的价值不仅限于服务 `coding-agent` ，还可以支撑更多垂直场景的 Agent 建设：

`component-standardizer-agent` ：组件标准化迁移

适用场景

存量旧代码迁移到标准化组件

从零开始接入标准化组件

实践效果

倒计时组件、价格组件等核心组件的批量迁移，原本需要逐个文件人工替换的工作，现在可以通过 Agent 自动完成代码改写和验证。

组件类型	迁移效果
倒计时组件	
价格组件	

`frontend-qna-agent`：业务知识智能问答

适用场景

日常技术答疑，覆盖会场、百补直播间、百补小购物车等业务场景

实践效果

基于知识库的精准问答，避免了重复回答相同问题，新同学可以通过 Agent 快速获取业务上下文。



07 实践效果：从理论到落地

`coding-agent` 已经不再是理论上能够提效的玩具，而是在团队迭代中实际使用的辅助开发工具。以下是工程链路中的典型应用场景：

项目依赖批量升级

场景描述：全链路测试，纯逻辑变更

Agent 输入：页面 A 升级依赖 B 到最新 beta 版本，推送并发布测试页面

效果展示：执行步骤数据含业务数据，安全评估暂不对外开放

页面级代码生成

场景描述：基于页面描述、设计稿链接、截图等信息完成页面开发

Agent 输入：在本仓库完成需求开发。设计稿链接：XURL。备注：完整的页面开发，需要页面通顶！

效果展示：执行步骤数据含业务数据，安全评估暂不对外开放

原图	Agent 直出效果



知识库在线更新

场景描述：在线化更新业务知识库信息

效果展示：执行步骤数据含业务数据，安全评估暂不对外开放

08 总结与展望

现阶段的 `coding-agent` 探索还遗留了很多亟待解决的问题：

- D2C 场景覆盖：提升覆盖率，真正解决 90% 以上 UI 还原场景带来的重复性工作
- 知识库丰富度：持续丰富知识库体系，打破跨业务交付的壁垒
- 交互方式升级：从自然语言 → 富文本、Web 可视化
- 目标用户扩展：从面向开发 → 面向产运研
- 交付粒度提升：从自然语言到代码 → 自然语言到完整页面交付
- 更多业务场景：探索更多可能性...

我们的目标是完成端到端完整的交付链路，所以现阶段的结果也只是起点，AI 每天都在以超乎想象的速度进步，`coding-agent` 也会更加强大！

09 团队介绍

本文作者程禄，来自淘天集团-天猫技术团队。百亿补贴与聚划算，是淘天集团面向价格敏感用户和品牌商家的核心营销阵地，也是我们直面市场竞争最前沿的主战场。我们不断刷新技术能力，支撑百亿补贴成为淘天过去三年里增长最快的业务。当前，我们也正在用 AI 重构整个运营流程：从超链半托管、招商审核，到补贴，全面推动从“人工配置”向“AI 自治”演进。同时，我们也正积极探索 AI 对全栈研发流程的深度重构 —— 从需求分析、接口设计、代码生成，到测试验证与部署上线，逐步构建端到端的 AI 原生开发范式。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 8392

留言

写留言

还没有留言，来写首评